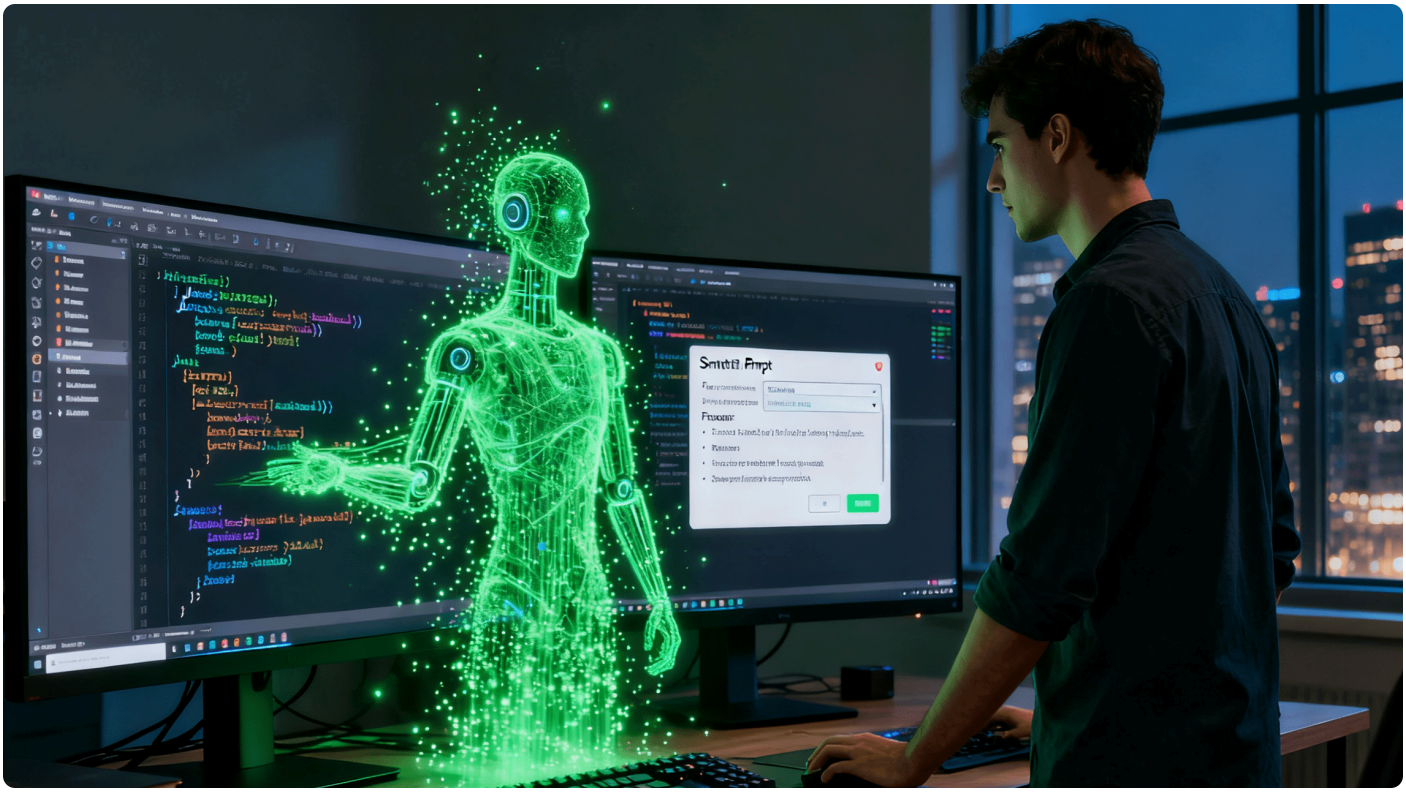
02 | 核心引擎：AI原生开发第一性原理——规范驱动开发

Tony Bai · AI 原生开发工作流实战



你好，我是 Tony Bai。

在上一讲，我们共同建立了一个“AI– 开发者集成成熟度模型”，并清晰地描绘了开发者在 AI 原生时代的角色演进：从代码的生产者，转变为**规范的设计者**、**工作流的指挥家**和**质量的治理者**。

你可能会好奇，这三个新角色中，为什么“规范设计师”排在第一位?

答案很简单：在 AI 原生开发范式中，一份高质量的、可被机器精确理解的规范（Specification），是驱动整个开发工作流的起点和核心引擎。没有它，AI 的强大能力就如同无的之矢，无法精准地作用于我们的工程目标。

所以，今天这一讲，我们将深入整个专栏最核心的方法论：规范驱动开发（Spec–Driven Development, SDD）。你将理解它为何是 AI 原生开发的“第一性原理”，并学会运用它的核

心产物——`spec.md`、`plan.md` 和 `tasks.md`，将一个模糊的软件构想，一步步精确地“编译”成高质量的实现。

软件开发中“失落的翻译”问题

在深入 SDD 之前，我们必须先直面一个困扰了软件行业数十年的根本矛盾：意图（Intent）与实现（Implementation）之间的鸿沟。

在传统的开发流程中，我们是这样工作的：

1. 产品经理将业务构想，翻译成一份自然语言描述的 PRD（产品需求文档）。
2. 架构师和开发者阅读 PRD，进行“人脑编译”，将其翻译成技术设计文档。
3. 开发者再次进行“人脑编译”，将设计文档翻译成一行行具体的代码。

这个过程充满了“**失落的翻译**”。每一次“翻译”，都不可避免地引入了信息的损耗、歧义和主观臆断。PRD 中的一句“界面需要响应迅速”，到了开发者这里，可能被理解成“API 响应时间小于 500ms”，也可能被忽略。

更糟糕的是，这些文档（PRD、设计文档）与最终的“真理之源”——代码，是**完全脱节**的。一旦项目开始迭代，代码飞速演进，而文档却被遗忘在角落，迅速腐烂，变成“代码考古”的障碍。

几十年来，我们发明了敏捷、Scrum、UML 等无数方法和工具，试图弥合这条鸿沟，但收效甚微。因为我们始终默认了一个前提：**文档是代码的“指南”，而代码才是“真理”**。

而 AI 时代的到来，给了我们一个彻底颠覆这个前提的机会。

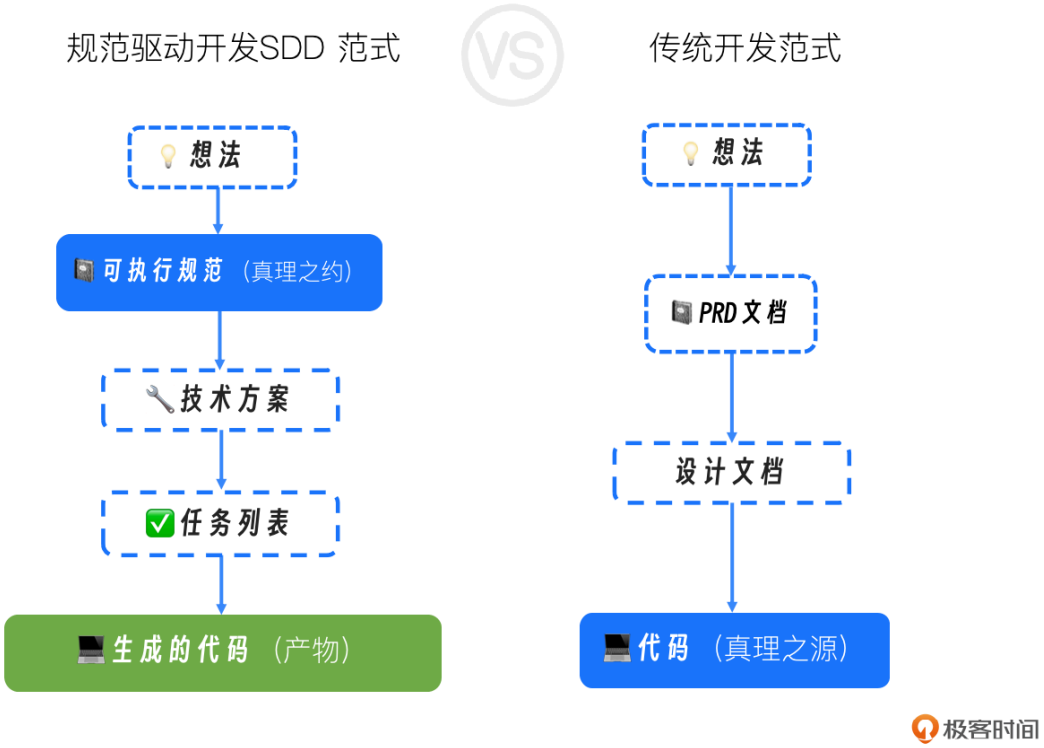
权力的反转：当规范成为“真理之源”

规范驱动开发（SDD）的核心思想，可以用一句话来概括：**实现一次“权力的反转”**。

在 SDD 范式中，不再是文档服务于代码，而是**代码服务于规范**。那份结构化的、无歧义的、可被机器理解的规范，取代了代码，成为了项目唯一的、至高无上的“真理之源”（Single Source of Truth）。

代码，则降级为这份“真理之源”在某个特定技术栈（比如 Go + Gin + PostgreSQL）下的一个自动渲染产物或编译结果。

我们可以用一张图来直观地感受这场“权力反转”带来的颠覆性变化：



在这场反转中：

维护软件的核心，从“修改代码”，变成了“演进规范”。

调试 Bug 的核心，从“修复错误代码”，变成了“修正产生错误代码的规范或方案”。

技术重构的核心，从“大规模迁移代码”，变成了“基于同一份规范，生成一个全新技术栈的实现”。

这听起来很美好，但要实现它，就要求我们的“规范”必须具备一个前所未有的特性：**可被机器精确理解和执行**。这就引出了一个贯穿本讲的核心隐喻。

编译意图：AI Agent 扮演的新角色

在讲解具体流程前，我必须先解释清楚一个关键的隐喻：在 SDD 中，我们常说 AI Agent 在“编译”我们的想法。

这里的“编译”并非传统意义上将高级语言（如 Go）转换为机器码。在 AI 原生开发中，“编译”指的是一个由 AI 驱动的、将高层级、模糊的、非结构化的人类意图，逐步转换、细化，并最终固化为低层级、精确的、结构化的机器可执行指令（即代码）的全过程。

在这个过程中，AI Agent 扮演了多个“编译器”的角色：

- **需求编译器：**将你用自然语言描述的模糊想法，“编译”成一份结构化的、无歧义的需求规范（`spec.md`）。
- **方案编译器：**将需求规范与你的技术约束（如使用 Go 语言）相结合，“编译”成一份详尽的技术实现蓝图（`plan.md`）。
- **任务编译器：**将技术蓝图，“编译”成一份带依赖关系的、原子化的任务指令集（`tasks.md`）。
- **代码编译器（生成器）：**最终，它根据任务指令集，生成最终的可执行代码。

理解了这个隐喻，你就能明白，我们在 SDD 中所做的一切，都是在为 AI 的“多阶段编译”过程提供高质量的“源代码”（即规范），并监督每一步“编译”的结果。

SDD 工作流的四个通用阶段

现在，让我们抛开任何具体的工具，来抽象地看一下规范驱动开发（SDD）通用的工作流。它通常分为四个阶段，每个阶段都产出一份愈发具体的标准化文档。

第一阶段：意图定义

- **目标：**澄清并固化“做什么”（WHAT）和“为什么做”（WHY）。

输入：开发者或产品经理提出的高层级、模糊的自然语言想法。

核心活动：人机协作进行头脑风暴，挖掘边缘场景，澄清模糊地带，定义验收标准。

输出产物：`spec.md` 需求规范。这份文档的核心是**完全与技术实现解耦**，它只关心用户故事、功能需求和成功标准。它是后续所有阶段的唯一输入和“宪法”。

第二阶段：技术规划

目标：决定“如何做”（HOW）。

输入：`spec.md` 和开发者提供的技术栈约束。

核心活动：AI Agent 基于规范，结合自身的工程知识和项目已有的“宪法”（constitution.md），进行技术选型、架构设计、模块划分、API 契约定义。

输出产物：`plan.md`（技术方案）及其附属文档（如 data-model.md、api-spec.json）。这份方案将业务需求精确地映射到技术实现上。

第三阶段：任务分解

目标：将“如何做”分解为“一步步怎么做”（ACTIONS）。

输入：`plan.md` 及其所有附属设计文档。

核心活动：AI Agent 分析技术方案，将其拆解成一系列具体的、原子化的、可执行的开发任务。关键在于，它需要识别任务之间的依赖关系和可并行点。

输出产物：`tasks.md`（任务列表）。这份文档是 AI Agent 的“待办事项清单”，其格式和内容必须是机器友好的。

第四阶段：自动化实现

目标：完成所有任务，生成最终产物。

输入：`tasks.md`。

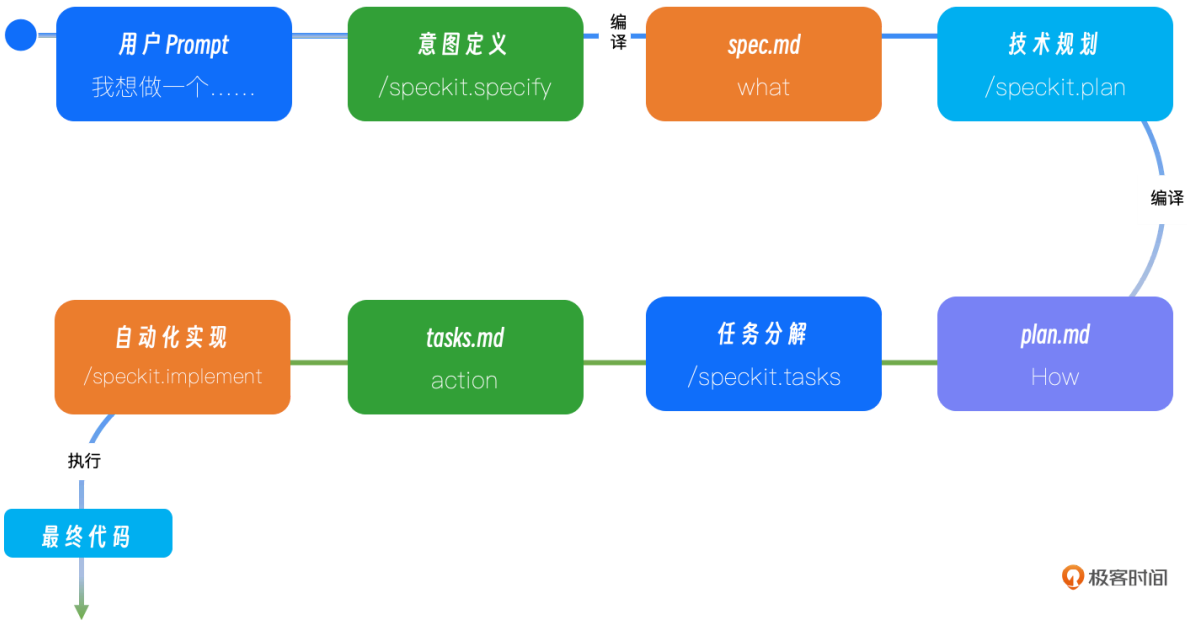
核心活动：AI Agent 严格按照 `tasks.md` 的指令，逐一执行任务，包括创建文件、编写代码、运行测试等。开发者在此阶段的核心职责是监督、审批高风险操作，以及对最终结果进行验收。

输出产物：可运行的软件代码、测试用例、文档等。

这四个阶段，构成了一个从“抽象意图”到“具体实现”的完整逻辑链条。

一个具体实现：以 GitHub 的 spec-kit 为例

理论总是有些枯燥，现在让我们看看一个真实的、业界领先的开源项目——GitHub 的 `spec-kit` 是如何通过一系列标准化的 **Slash Commands**（斜杠指令）来实现上述四个阶段的。`spec-kit` 是规范驱动开发思想的一个杰出实现，而 Claude Code 等 AI Agent 正是执行这套流程的理想工具。



让我们以 `spec-kit` README 中的“照片相册应用”为例，走一遍这个流程：

1. 意图输入（/speckit.specify）：你向 AI Agent 输入一个自然语言的想法：“`Build an application that can help me organize my photos...`”。
2. “编译”成规范（spec.md）：AI Agent 扮演产品经理的角色，与你互动（或基于内置模板），将你的模糊想法“编译”成一份结构化的 `spec.md`。同时，它会自动为你创建一个新的 Git 分支，如 `001-photo-albums`。

3. 技术选型（/speckit.plan）：你进入技术决策阶段，告诉 AI：“The application uses Vite...vanilla HTML, CSS, and JavaScript...SQLite database.”。
4. “编译”成方案 (plan.md)：AI Agent 扮演架构师的角色，将技术选型与 `spec.md` 的需求结合，生成一份详尽的 `plan.md`。
5. 生成任务列表 (/speckit.tasks)：你下达指令，不需额外参数。
6. “编译”成“字节码” (tasks.md)：AI Agent 扮演技术组长的角色，它读取 `plan.md`，将其分解为上百个具体的、带依赖关系和并行标记的原子任务，生成 `tasks.md`。
7. 执行实现 (/speckit.implement)：你下达最终执行指令。
8. “执行”字节码（Code）：AI Agent 像一个勤奋的程序员，严格按照 `tasks.md` 的指令列表，逐一完成编码、测试、创建文件等所有工作，最终交付可运行的应用程序。

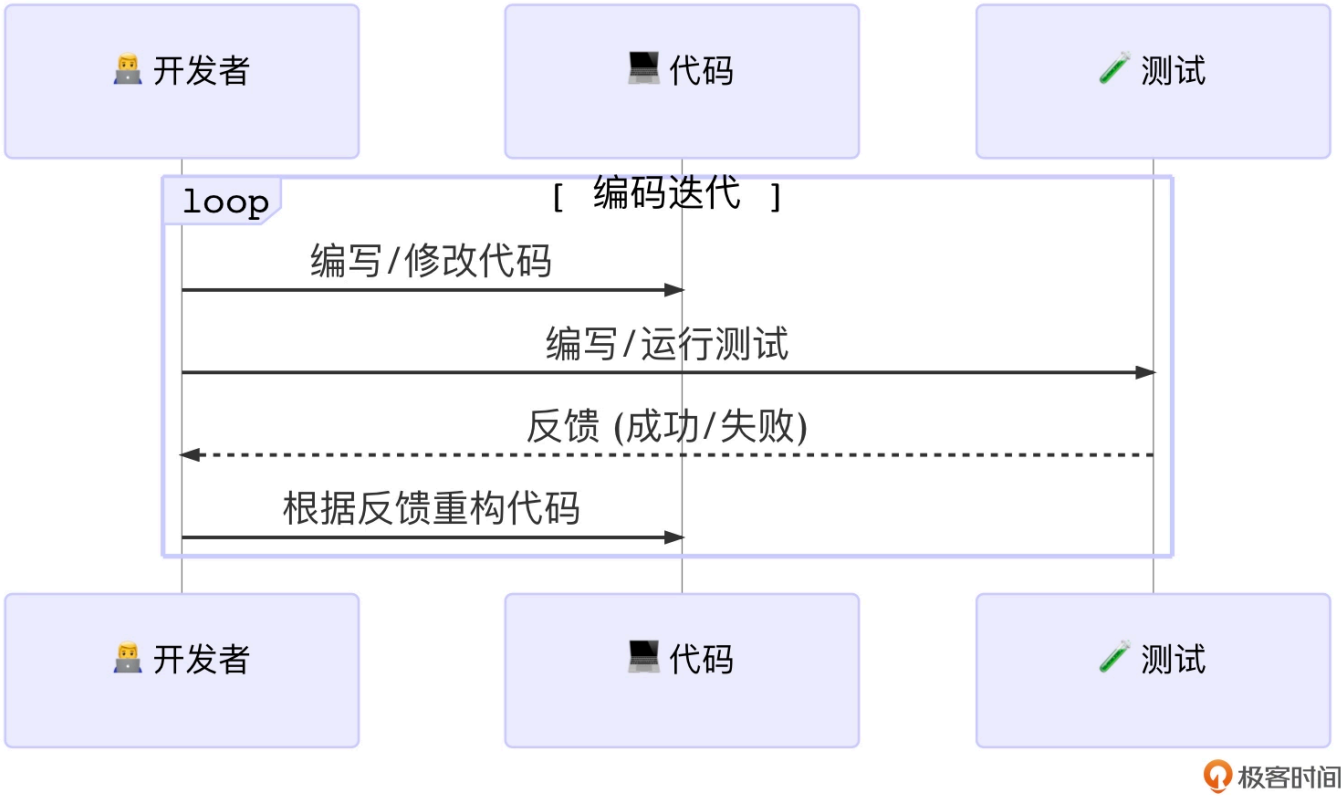
通过这个例子，你可以清晰地看到，像 `spec-kit` 这样的框架，就是将 SDD 的通用思想，固化为了一套可执行、可重复的工程实践。

新范式下的开发循环

SDD 的引入，不仅改变了项目的启动流程，更从根本上重塑了我们日常的开发迭代循环。

传统开发循环：以“代码”为中心

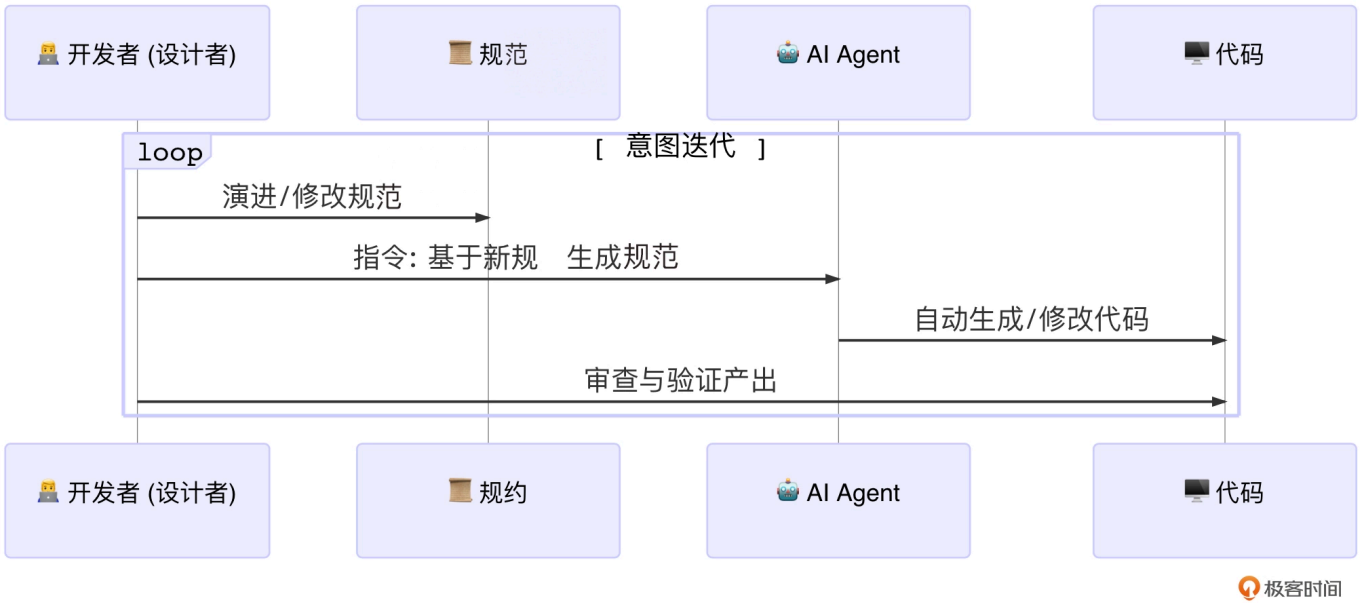
我们熟悉的敏捷开发，其核心是一个以代码为中心的 Code -> Test -> Refactor 循环。



这个循环非常高效，但它的半径很小，始终围绕着“实现”这个层面。当需求发生较大变更时，这个小循环往往需要被打破，回到更上游的设计阶段，成本很高。

AI 原生开发循环：以“规范”为中心

而在 SDD 范式下，开发循环被提升到了一个更高的维度，变成了一个以“规范”为中心的 Spec -> Generate -> Validate 循环。



在这个新循环中，开发者的主要工作不再是直接编写和重构海量代码，而是：

- **演进规范：**思考并修改 `spec.md` 来响应需求变化。
- **触发生成：**指挥 AI Agent 基于新的规范重新生成实现。
- **审查验证：**验收 AI 的产出是否符合规范的意图。

这个循环的半径更大，它直接作用于“意图”层面。它的优势在于，**迭代的成本大大降低了**。因为 AI 接管了最耗时、最易出错的“代码翻译”环节，我们可以用前所未有的速度，试验不同的需求、不同的架构，甚至不同的技术栈。

SDD 为什么是 AI 原生开发的未来?

理解了 SDD 的理念和流程，你就能明白，它为何不仅仅是一种新方法，而是与 AI Agent 能力天然契合、相辅相成的核心引擎。

它有助于解决 AI 的“模糊性”难题：大模型本质上是基于统计的，对模糊指令的理解存在不确定性。SDD 通过结构化的规范，将模糊的自然语言，转化为 AI 可以精确执行的“机器语言”，极大地提升了输出的可靠性。你不再需要祈祷 AI“猜对”你的心思，而是通过一份严谨的规范来精确地“编程”AI 的行为。

它加速了软件的“迭代”循环：在传统模式下，需求变更意味着昂贵的人工代码修改成本。在 SDD 模式下，“业务逻辑”的核心被固化在更高层次的 `spec.md` 中。当需求变化时，我们只需修改规范，然后快速地“重新编译”整个实现。软件开发的迭代速度，从“周 / 天”级别，提升到了“小时 / 分钟”级别。这使得真正的敏捷——快速响应变化，成为了可能。

它释放了 AI 的“并行”潜力：`tasks.md` 中的并行标记 `[P]`，为多个 AI Agent 协同工作，或单个 AI Agent 并发执行任务提供了可能。这突破了人类开发者线性工作的瓶颈，为实现真正的“10 倍效能”打开了大门。

它创造了真正的“活文档”：当文档（规范）成为驱动代码生成的唯一源头时，“文档与代码不一致”这个困扰我们几十年的毒瘤，便被彻底根除了。规范即文档，文档即代码，代码即实现。

它最终赋能了“开发者”：SDD 将开发者从繁琐、重复的实现细节中解放出来，让我们能将更多的智慧和精力，投入到更高价值的业务建模、架构设计和工作流优化中去。这完美地呼应了我们在上一讲中提到的开发者角色的三大转变。

当然，我们也必须清醒地认识到，在今天，要实现一套完全无缝、100% 自动化的规范驱动开发（SDD）流程，仍然存在着一定的门槛和挑战。这主要受限于当前大语言模型在超长链条、复杂逻辑推理上的稳定性，以及像 Claude Code 这类客户端智能体工具在任务执行的鲁棒性和上下文环境感知深度上的局限。

但这是否意味着 SDD 只是一个遥不可及的“空中楼阁”呢？

恰恰相反。**这正是我们现在系统性地学习和实践它的最佳时机。**

SDD 为我们描绘了一幅清晰的、通往 AI 原生开发终局的蓝图，它定义了人与 AI 在软件工程中最高效、最可扩展的协作接口。我们今天学习的每一项技能——如何编写精确的规范、如何设计可被 AI 调用的工具、如何构建自动化的验证闭环——都是在为那个完全自主的 **Level 4** 时代的到来铺设最坚实的基础。随着 AI 大模型与智能体工具的飞速迭代，我们有理由相信，完全基于 SDD 的软件工程范式，不仅是可能的，而且是必然的未来。

本讲小结：理解 SDD，是掌握 AI 原生开发的第一步

今天，我们深入了 AI 原生开发方法论的核心引擎——规范驱动开发（Spec-Driven Development）。这不仅是一种新的技术，更是一种足以重塑我们开发哲学的“第一性原理”。

首先，我们直面了传统开发中“意图”与“实现”之间的鸿沟，即“**失落的翻译**”问题。接着，我们学习了 SDD 的核心思想——权力的反转，即规范取代代码，成为唯一的“真理之源”。并引入了“编译意图”这一核心隐喻，将 AI Agent 定位为将人类意图逐步编译为代码的多阶段编译器。

我们将 SDD 的工作流抽象为四个通用阶段：**意图定义、技术规划、任务分解和自动化实现**。并以 GitHub 的 `spec-kit` 为例，展示了这套通用流程在真实世界中的一个具体实现。我们还对比了新旧两种开发循环，揭示了从“以代码为中心”的 Code -> Test -> Refactor，到“以规范为中心”的 Spec -> Generate -> Validate 的根本性转变。

最后，我们理性地探讨了 SDD 的现状与未来。虽然完全实现它尚有挑战，但我们明确了它就是通往 AI 原生开发终局（Level 4）的蓝图和必经之路。掌握 SDD，是你从一个 AI 工具的“使用者”，转变为一个 AI 工作流“指挥家”的必经之路。它为你提供了一套强大的思想武器，让你能够结构化地、系统化地驾驭 AI 的强大能力。

下一讲，我们将回归更广阔的视野，扫描当前命令行 AI Agent 的生态全景。我将带你了解除了 Claude Code，还有哪些值得关注的工具，并最终解释清楚，为什么我们选择 Claude Code 作为本专栏后续所有内容以及实战的核心载体。

思考题

请你回想一个你最近参与开发的功能。尝试用我们今天学习的 SDD 三大产物的思想来解构它：

1. 你能否用 `spec.md` 的原则（纯粹的 WHAT 和 WHY，无技术实现），为这个功能写一个简要的用户故事和几个验收标准?
2. 在你当时的技术方案中，哪些决策可以被归入 `plan.md`?

3. 你实际的编码过程，能否被拆解为一系列像 `tasks.md` 中那样原子化的、带依赖关系的任务？

这个练习将帮助你更好地理解 SDD 思想，并发现它与你当前工作流的异同。欢迎在评论区分享你的尝试和思考，与我一起交流讨论，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (41)

最新　精选



望星空 置顶

2025-11-28 来自广东

老师，写的非常好! 读完后感觉 SDD 真牛。我们已经在一个前后端一起的项目中，用AI实现了很多需求，但是效果达不到预期。请问该如果接入 SDD?

作者回复: 嗯嗯，后面有讲解。

共 2 条评论 >　　2

罗烽

2025-11-21 来自北京

老师，咨询一下，如果使用spec生成的代码，在测试阶段发现实现bug，我是应该修改spec相关文档，让其自主修复？还是直接手动修复代码?

作者回复: 非常好的问题，很多使用SDD，基于cc等coding agent的小伙伴都有类似问题。

我的想法是取决于 Bug 的性质：是“没想清楚”还是“没写对”。

如果是“意图”错了（没想清楚）：修改 Spec，比如你发现业务逻辑本身有漏洞，或者漏掉了某个边缘场景。这样，最好修改 spec.md，然后让 Agent 重新生成。

如果Spec 写得很清楚，但 AI 生成的代码有逻辑错误、语法错误或性能问题，那么让 Agent 修复代

码。这时 Spec没错，你应该把错误日志、失败的测试用例反馈给 Agent，指令它：“实现有误，请根据这个错误信息修复代码”。

总之，一个原则尽量把握住：永远维护“意图”的单一来源。 只有当 Bug 源于意图偏差时才改 Spec，否则让 Agent 去修代码。

共 3 条评论 >

48



grok

2025-11-22 来自美国

tony大佬，你对半路出家做开发的同学有什么建议(比如本专业是化工)？记得几年前手写代码时还听到有人劝，你应该自学数据结构/算法/架构/blabla

现在cursor三下五除二 代码全出来了。更没动力自学那些计算机系的基础课了。

您怎么看？给兄弟们点建议？

作者回复: 对于半路出家的同学，像Cursor 这类AI工具是巨大的福音。先别纠结底层，利用 AI 快速把产品/功能做出来，建立信心和正反馈，这是最关键的第
一步。

但接下来，就进入深水区了。AI 能“三下五除二”生成代码，但结构是否优秀，代码对错、性能优劣、安全隐患需要你来把关。换句话说，在这个AI时代，基础可能不再是“敲门砖”了，但它可能是你职业发展的“天花板”。

此外，有了AI也得继续学习，在实战中学，在AI的帮助下学习。比如可能不需要像以前那样枯燥地啃大部头的书籍了，把 AI 当成最好的私教。在实
战中遇到不懂的，反过来补基础，效率可能会高得多。

AI 替你由 0 到 1 搬砖，而需要你负责由 1 到 100 的设计与把关。如果你把关的底气不足，那就是学习的最好动机。

共 4 条评论 >

42



s(t)

2025-11-21 来自上海

有个疑问，就算在 ssd 模式下失落的翻译问题也是会存在的吧？只是把意图定义这一步放在明面上了，规定你必须尽可能考虑每个 case，以及沟通好每个需求的本质目的等。但要是沟

通的不到位，就算有 spec.md，还是会有信息损耗的问题吧。意图定义这一步是为了让我们把更多的视角精力放在明确意图上，避免像之前一样只顾直接上手写代码。

作者回复: 是的。SDD 并不能从物理上“消灭”信息损耗，而是将损耗控制在了成本最低的阶段。传统开发是“产品经理想 -> PRD -> 研发看 -> 代码”。信息损耗往往在代码写完甚至上线后才被发现，修改成本极高（重写代码、重测）。
SDD 将确认环节提前到了 spec.md 阶段。在这里发现沟通不到位（损耗），修改的只是一段 Markdown 文本，而不是几千行代码。

启发我了，SDD是不是可以理解为“损耗左移”？

“失落的翻译”永远存在，但 SDD 让我们在动工之前就不得不把翻译做准，而不是等到完工之后才发现翻译错了。这更像是用“思考的成本”置换了“返工的成本”。

共 2 条评论>

31



Z12
2025-11-22 来自广东

老师，我想问一个问题，这些规范文档是要在每一次交互中输出给 AI 吗？如果一个庞大的项目，需求文档是很大的，上下文和 token 的限制会不会导致难以实现这个工作流呢?

作者回复: 这就涉及到像Claude Code这样的ai agent的工作原理了，实际上也是基于大模型的ai应用的开发原理。服务端大模型原生是没有记忆的（也有些模型厂商通过infra给大模型外置了记忆，降低会话通信开销和推理开销），上下文全靠客户端投喂。因此ai agent运行时token的耗费还是蛮巨大的。

一些技巧就是每个session的上下文边界要控制好。

每个任务一个session。不要在一个session里无穷无尽的交互下去。

21



smallFive55
2026-01-06 来自湖南

老师，我也一直在思考怎么使用 AI 规范化 标准化的编程，看了课程更加坚定了想要规范化 AI 编程的想法。对于 SDD 我的思考是： 在超过 20 人的技术团队中，目前无法把控所有开发人员都按照标准流程进行去修改 spec.md ，对于一些处理非常简单的错误，很有可能直接上手修改或者直接指示 AI 修改代码。这样一来 spec.md 就不完全正确了～

作者回复: SDD 在多人团队落地的确会有很多挑战！

不要指望所有小修小补都去改 Spec。毕竟“人性本懒”:)。区分一下，必须走 Spec 变更流程的，可以靠 CI/CD 流程卡点，没 Spec 不让合并。同时
，对于极其简单的 Bugfix/小优化，也允许直接改代码，但要求Commit Message 清晰。

另外看看是否加一个兜底方案，那就是让 AI 帮你“反向同步”。定期（或 Release 前）运行一个任务： 让 AI 读取所有新的代码变更（git diff）
，反向更新 spec.md。可以下达这样的指令：“根据最新的代码实现，同步更新 spec 文档，保持一致性。”。可以看看效果如何。

共 2 条评论>

12



单向地铁

2025-11-25 来自重庆

老师您好，在体验spec-kit的过程中，我有两个疑问想想请教：

我感觉spec-kit像是一辆“笨重的坦克”，它的执行过程非常线性。一旦进行到了后续阶段，如果发现之前的spec或者plan需要优化或补充，哪怕是很小的补充，它也会自动新建一个分支，并重新开始，导致使用起来感觉灵活性较差。

另外，我非常认同SSD的理念，但我也好奇：通过SSD会生成大量的文档，当文本量很大，细节很多的情况下，AI写出的代码能否准确地关注并执行每一个细节吗?

作者回复: 好问题。

目前的 spec-kit 作为 SDD 的参考实现，我同样感觉偏向于“全新特性开发”的线性流程，在灵活迭代上存在局限。

但这是spec-kit的问题，并非SDD思想的问题。在实际操作中，我们不需要被 spec-kit 的 CLI 框死。

在本专栏后续讲解工作流时，我也没有使用spec-kit，而是手动来实现这个流程。一旦spec.md, plan.md, tasks.md生成后，这些 .md 文件就是你的资产，你可以像维护代码一样，灵活地手动或让 AI 局部更新它们，而不必每次都触发“重建分支”的重型操作。

另外关于“长上下文”挑战和AI 的注意力衰减问题，其实SDD 的精髓在于分层. spec.md 讲业务，plan.md 讲架构，tasks.md 讲步骤。AI 其实不需要一次性关注所有细节。 在执行 tasks.md 中的某一

个任务时（比如 Task 5），你只需要把 Task 5 的描述、相关的 spec 片段和当前涉及的代码文件喂给 AI。

目前，SDD 的工具（如 spec-kit）目前还在早期阶段，稍显笨重也是正常的。

共 3 条评论 >　　11



Josh
2026-01-07 来自北京

大佬讲的很清楚，但我的切实问题是，如何用来AI原生开发工作流解决存量工程的迭代效率？挺大的一个工程十几号人迭代了三年多，如何用这套流程用一半的人力能撑住日常的需求迭代。

作者回复: 这个问题挺大，可能也超出了该专栏可以回答的范畴。

团队情况不同，没有普适的方法和工作流。

不过，可以考虑从下面几个切入点逐步尝试：

– 让ai帮助“读存量代码”，遇到 Bug 或需求，先让 AI 解释这段遗留代码的逻辑（@file explain）。这能把排查时间从小时级压缩到分钟级，让新人（或更少的人）能快速上手老模块。
– 让ai写测试，建立存量项目的“质量网”。这样团队才敢于改代码。
– ai承接苦力活，针对高频重复模式，比如增删改查、DTO 转换、文档工作等。可封装专门的 Slash Command 或 Skill。把“填空题”交给 AI，人只负责审核。

先用ai切入上面这些工作，然后再逐步根据你们的情况形成你们的特色工作流，而不是一上来就sdd工作流。

共 2 条评论 >　　10



house
2025-11-21 来自广东

在软件系统开发迭代过程中，尤其是有状态系统软件设计，系统积累了很久的用户数据，在新功能迭代完全依赖agent去自动生成 会不会存在问题？

作者回复: 对于有状态系统，新功能的迭代往往伴随着 Schema 变更和存量数据迁移。AI Agent 如果只看当前需求，很容易生成“看似完美但无法兼容旧数据”的代码，导致各种问题。

在 SDD 模式下，不能只给 Agent 新功能的 Spec，必须注入相关的上下文。比如注入Schema ， 利用 @schema.sql 让 Agent 充分理解现有数据结构。再比如告诉 Agent “数据库变更必须向后兼容”、“必须生成对应的 migration 脚本”、“禁止删除现有字段”等。

另外人总是最后一道防线。在有状态系统的迭代中，Review 和 Checkpointing 尤为重要。对于涉及数据变更的操作，必须人工介入审查 Migration 脚本，并利用沙箱环境验证数据迁移的安全性。AI Agent负责生成，但人需要负责为数据安全兜底。

9

公号-技术夜未眠
2025-11-29 来自四川

老师好，如果利用SDD，面对的是遗留系统上进行新功能开发，这种场景的最佳实践是什么？以及如何能解决以下问题?
我想到的是会遇到以下问题:
(1) 新功能是需要基于/调用现有遗留系统的组件提供的API进行功能开发，而非采用重新实现的方式，Agent如何才能做到准确的调用?
(2) 需求内容往往会含有特定行业/应用的“黑话”、“行话”等私有的内部术语描述，而大模型一般是无法进行理解的，Agent如何才能准确理解意图并推进后续步骤执行?
谢谢老师回答

作者回复: 问题一：核心还是在于“上下文注入”。

提供接口文档： 如果有遗留系统组件的 Swagger/OpenAPI 文档，直接用 @ 喂给 Agent。

提供示例代码： 找一段现有的、调用了相关 API 的老代码，用 @ 喂给 Agent 作为 Few-Shot 示例。Agent 的模仿能力极强，它会照猫画虎。

另外，也可以讲遗留系统代码（如果可以的话）提供给ai agent，像Claude Code可以自己去理解 API 用法。

问题二：

你可以在项目级 CLAUDE.md 中专门开辟一个 ## Terminology 章节，把“黑话”翻译成通用的技术语言（例如：“‘打桩’= 创建 Mock 对象”）。或者
建立docs/term.md，然后在Claude.md中引用。 作为长期记忆： 这样每次启动会话，coding Age

nt 都会自动加载这份“字典”，从而能听懂你的行话。

共 2 条评论>

7